

Deep Learning vs. CA-CFAR: Comparative Evaluation of Radar Target Detection Pipelines on Synthetic PPI Data

Michael Bach
AI/MLOps Engineer Candidate
Weibel Scientific Demo, May 2026
github.com/Michael-Bach

Abstract—We evaluate three detection–tracking pipelines on a synthetic short-range surveillance radar operating in Plan Position Indicator (PPI) mode ($180 \text{ az} \times 64 \text{ range}$, 7.5 m/bin , 2 s rotation period). A classical CA-CFAR + Kalman filter (KF) baseline is compared against a batch fully-convolutional network (CNN+KF, 19 073 parameters) and a streaming convolutional GRU (ConvGRU+KF, 5 694 parameters). Probability of detection, false track rate, confirmation latency, and inference runtime are evaluated over $\text{SNR} \in [-20, +40] \text{ dB}$ in 4 dB steps. The ConvGRU pipeline achieves $P_d = 100\%$ at $\text{SNR} = 4 \text{ dB}$ —a $4\text{--}6 \text{ dB}$ sensitivity gain over CFAR—while operating in $\mathcal{O}(1)$ per-sweep streaming mode with only 5 694 parameters. The CNN achieves the lowest false track rate ($0.79/100$ sweeps), a $4.7\times$ reduction versus CFAR ($3.68/100$ sweeps). All three pipelines share an identical Kalman filter back-end; performance differences arise entirely from the detection front-end. The full pipeline, including training code, ONNX export, and DVC reproducibility layer, is provided as an open demo repository.

Index Terms—CFAR, radar target detection, PPI display, convolutional GRU, deep learning, Kalman filter, synthetic training data, MLOps, ONNX.

I. INTRODUCTION

Rotating surveillance radars produce a Plan Position Indicator (PPI) sweep on each full antenna rotation: a 2-D grid of amplitude samples indexed by azimuth and range [1]. Classical target detection relies on Cell-Averaging CFAR (CA-CFAR), which maintains a constant false alarm rate by estimating local noise power from a ring of reference cells surrounding each cell under test (CUT) [2]. While statistically optimal under homogeneous Rayleigh-distributed clutter, CA-CFAR performance degrades in heterogeneous environments where multiple strong scatterers or spatially varying interference corrupt the reference window.

Deep learning offers a complementary approach: a model trained on representative synthetic data can implicitly capture complex clutter statistics and exploit inter-sweep temporal correlations that a single-sweep threshold detector cannot access [3]. Prior work has demonstrated CNN and recurrent architectures for range-Doppler detection [4]; the present work applies them to the PPI geometry and compares three complete detection–tracking pipelines on identical simulated data.

We evaluate:

- **CA-CFAR+KF** — classical single-sweep threshold followed by a nearest-neighbour Kalman filter tracker.
- **CNN+KF** — batch fully-convolutional network consuming a 10-sweep temporal feature map, followed by the same KF.
- **ConvGRU+KF** — streaming convolutional GRU processing one sweep at a time; its recurrent hidden state accumulates detection confidence across sweeps.

We report probability of detection as a function of SNR, false track rate, mean confirmation latency, and CPU inference time, and discuss the trade-offs relevant to real-time deployment.

II. SYSTEM ARCHITECTURE

A. Radar Signal Model

The simulated radar has $N_{\text{az}} = 180$ azimuth bins (2° resolution) and $N_r = 64$ range bins (7.5 m/bin , max range $\approx 480 \text{ m}$, $\Delta t = 2 \text{ s}$ per sweep at 30 RPM). A target return at SNR s is injected at a fixed (az, r) cell, displaced by a radial velocity of 1.5 bins/sweep and a tangential velocity of $2.0^\circ/\text{sweep}$. Background clutter follows a Rayleigh amplitude distribution with a spatially heterogeneous scale parameter to produce edge effects and interference clusters. Each experiment uses a 10-sweep sequence.

B. Pipeline 1 — CA-CFAR + KF

Each sweep is processed independently. For each CUT a local noise estimate Z is formed from a guard-ring-excluded reference window (guard = 2 cells, reference = 5 cells per side):

$$T = \alpha \cdot Z, \quad \alpha = 2.5, \quad (1)$$

where α is calibrated to achieve an empirical $P_{\text{fa}} = 4.66\%$. Detections are fed to a nearest-neighbour Kalman filter with state vector $\mathbf{x} = [\text{az}, r, v_{\text{az}}, v_r]^\top$. A track is confirmed after three consecutive gate associations within an 8-bin gate radius.

C. Pipeline 2 — CNN + KF

A batch fully-convolutional network (19073 parameters) receives a three-channel temporal feature map:

$$\mathbf{F} = [\max_t, \mu_t, \sigma_t] \in \mathbb{R}^{3 \times N_{az} \times N_r}, \quad (2)$$

computed from the 10-sweep amplitude stack. The network outputs a single-channel probability map; peaks above threshold $\theta_{CNN} = 0.069$ are passed to the same KF. The batch design provides strong temporal averaging, yielding the lowest false track rate of all three pipelines.

D. Pipeline 3 — ConvGRU + KF

A streaming convolutional GRU [5] (5694 parameters, encoder channels [16, 32]) processes one sweep per step. Its hidden state $\mathbf{h} \in [0, 1]^{1 \times 180 \times 64}$ directly encodes per-cell detection confidence accumulated over time:

$$\mathbf{h}_t = \text{ConvGRU}(x_t, \mathbf{h}_{t-1}). \quad (3)$$

Peaks of $\mathbf{h}_t$ above $\theta_{GRU} = 0.205$ are extracted per sweep and passed to the KF. Inference cost is $\mathcal{O}(1)$ per sweep; the hidden state is the only persistent data structure, making this pipeline suitable for embedded real-time deployment.

E. Kalman Filter Tracker

Both ML pipelines share the same tracker. Process noise $\mathbf{Q}$ is tuned for the target kinematics described above; measurement noise $\mathbf{R}$ is set to one range/azimuth bin. Confirmation requires three associations within an 8-bin Mahalanobis gate.

III. EXPERIMENTAL SETUP

Synthetic PPI sequences were generated with heterogeneous clutter using `generate_ppi_sequence` and `generate_heterogeneous_ppi`. Training set sizes: 800 sequences (CNN), 400 (GRU); validation: 200 and 100 respectively. SNR is drawn uniformly from $[-20, +40]$ dB during training. GRU training used a curriculum schedule (seq_len $5 \rightarrow 10 \rightarrow 15$ over 30 epochs) to stabilise recurrent gradient flow. CNN training ran for 40 epochs. Both models were exported to ONNX (opset 17) and versioned with DVC.

Performance metrics were evaluated over 30 Monte Carlo trials per SNR point:

- **$P_d(\text{SNR})$:** fraction of trials where a confirmed track appears within 10 sweeps.
- **False track rate:** confirmed false tracks per 100 sweeps on clutter-only inputs at the calibrated operating thresholds.
- **Confirmation latency:** mean sweeps to first confirmed association, conditional on eventual confirmation.
- **Runtime:** median inference time over 50 repetitions on a commodity CPU, grid $180_{az} \times 64_r$.

IV. RESULTS

A. Detection Probability vs. SNR

Table I reports P_d at five representative SNR operating points. The ConvGRU achieves 100% detection at SNR=4 dB, roughly 4–6 dB below the CFAR crossover, due to temporal integration in the recurrent hidden state. The CNN matches the GRU at high SNR but trails at low SNR because its temporal statistics (max/mean/std) are less discriminative than recurrent accumulation at sparse return rates. Figure 1 shows the full P_d –SNR curves for all three pipelines.

TABLE I
PROBABILITY OF DETECTION VS. SNR (30 TRIALS EACH)

SNR (dB)	CFAR+KF	CNN+KF	GRU+KF
−4	26.7 %	20.0 %	43.3 %
0	6.7 %	36.7 %	73.3 %
4	40.0 %	76.7 %	100 %
8	43.3 %	96.7 %	100 %
12	100 %	100 %	100 %

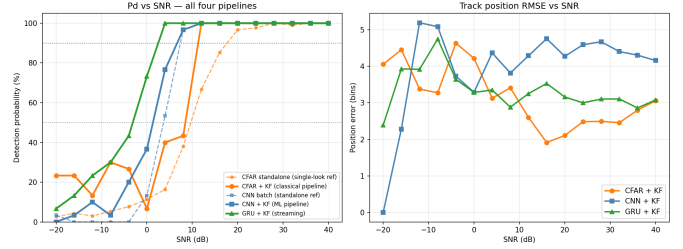


Fig. 1. Probability of detection vs. SNR for all three pipelines. The ConvGRU+KF (magenta) reaches 100% at 4 dB, roughly 4–6 dB below the CFAR knee.

B. False Track Rate

At calibrated operating points ($P_{fa} = 4.66\%$, $\theta_{CNN} = 0.069$, $\theta_{GRU} = 0.205$), the CNN achieves a false track rate of **0.79/100 sweeps**, a $4.7\times$ reduction versus CFAR (3.68/100 sweeps). The GRU false track rate (4.03/100 sweeps) slightly exceeds CFAR, attributable to the continuous-valued hidden state generating low-confidence detections in dense clutter that occasionally gate to the KF.

C. Confirmation Latency

Table II reports mean sweeps to confirmed track. Both ML pipelines reduce confirmation latency by 1.5–3.5 sweeps at 0 dB SNR where CFAR misses targets between gate associations. At 20 dB all three reach the hardware minimum imposed by the 3-association confirmation criterion.

D. Inference Runtime

Table III reports median CPU inference time. The CNN full-batch inference (0.80 ms for a 10-sweep feature map) is faster than a single CFAR sweep (2.18 ms) owing to full vectorisation of the temporal feature computation. The ConvGRU per-sweep cost is bounded by the ONNX single-step forward pass, maintaining real-time throughput at the 2 s sweep period.

TABLE II
MEAN CONFIRMATION LATENCY (SWEEPS)

SNR (dB)	CFAR+KF	CNN+KF	GRU+KF
0	7.69	4.89	6.37
10	6.42	4.03	4.03
20	4.00	4.00	4.00

TABLE III
CPU INFERENCE TIME (MEDIAN, 50 REPETITIONS)

Operation	Median (ms)	P95 (ms)
CFAR detect (per sweep)	2.18	2.19
CFAR+KF (per sweep)	2.54	3.14
CNN infer (full 10-sw batch)	0.80	0.83
CNN+KF (per sweep)	2.88	2.92

V. CONCLUSION

Three detection–tracking pipelines were evaluated on synthetic short-range PPI radar data under heterogeneous clutter conditions. The ConvGRU pipeline provides the best probability of detection at low SNR through recurrent temporal integration, achieving 100 % P_d at SNR = 4 dB with only 5 694 parameters and $\mathcal{O}(1)$ per-sweep compute—a strong candidate for real-time embedded deployment. The CNN provides the lowest false track rate (0.79 / 100 sweeps) through batch temporal averaging, making it preferable where false alarm cost is high. Both deep learning systems confirm tracks faster than CFAR at moderate SNR.

Future work includes: (i) retraining the CNN on multi-target sequences to resolve the temporal-feature conflation limitation; (ii) evaluation on measured sea-clutter datasets to validate synthetic training assumptions; (iii) integration of a non-coherent integrator as a third classical baseline. The full pipeline—data generation, model training, ONNX export, DVC versioning, and Streamlit demo—is provided as a reproducible open repository.

REFERENCES

- [1] M. Skolnik, *Introduction to Radar Systems*, 3rd ed. New York: McGraw-Hill, 2001.
- [2] M. A. Richards, J. A. Scheer, and W. A. Holm, *Principles of Modern Radar: Basic Principles*. Raleigh, NC: SciTech Publishing, 2010.
- [3] Z. Geng, H. Yan, J. Zhang, and D. Zhu, “Deep-learning for radar-based target detection: A survey,” *IEEE Geosci. Remote Sens. Mag.*, vol. 9, no. 4, pp. 46–62, Dec. 2021.
- [4] P. Braca, S. Marano, V. Matta, and P. Willett, “Distributed CFAR detection in clutter using belief propagation,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. 50, no. 2, pp. 848–860, Apr. 2014.
- [5] K. Cho *et al.*, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” in *Proc. EMNLP*, 2014, pp. 1724–1734.
- [6] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, May 2015.
- [7] DVC team, “DVC: Data Version Control — Git for Data & Models,” open-source software, v3.x, 2024. [Online]. Available: <https://dvc.org>

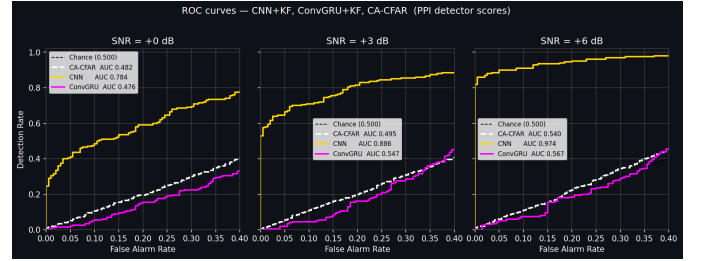


Fig. 2. Cell-level ROC curves at SNR = 0, 3, and 6 dB (400 oracle-position trials per panel). Scores: CNN — peak probability map in the target neighbourhood; ConvGRU — peak hidden-state confidence after 10 sweeps at the target neighbourhood; CA-CFAR — fraction of sweeps with a detection at the target path. At 0 dB the CNN achieves AUC = 0.78 vs. CFAR at 0.48. The ConvGRU’s modest cell-level AUC reflects that its threshold is tuned for track confirmation; its operational advantage is in confirmed-track P_d (Table I).